

**LAPORAN PRAKTIKUM
IMPLEMENTASI PERANGKAT LUNAK**

**MODUL 8
IMPLEMENTASI PROSES DESIGN OBJECT ORIENTED**

**DISUSUN OLEH :
AMMAR BAGAS FATHURRAHMAN WANTORO 2350081008**



**PROGRAM STUDI INFORMATIKA
FAKULTAS SAINS DAN INFORMATIKA
UNIVERSITAS JENDERAL ACHMAD YANI
TAHUN 2025**

DAFTAR ISI

DAFTAR ISI	1
BAB I. HASIL PRAKTIKUM	2
I.1 Latihan Implementasi Class Diagram	2
I.1.A. Source Code	2
I.1.B. Analisa	5
BAB II. TUGAS PRAKTIKUM	6
II.1 Implementasi Class Diagram	6
II.1.A. Source code	6
II.1.B. Analisis	9
BAB III. KESIMPULAN	10

BAB I. HASIL PRAKTIKUM

I.1 Latihan Implementasi Class Diagram

I.1.A. Source Code

NotYetException.java

```
package latihan;

public class NotYetException extends Exception{
    public NotYetException(String s){
        super(s);
    }
}
```

Student.java

```
package latihan;

public class Student {
    private String name;
    public Student(String name){

    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}
```

Assignment.java

```
package latihan;

public class Assignment {
    private double mark = -1;
    public Assignment(){}
    public double getMark() throws NotYetException {
        if (mark== -1) {
            throw new NotYetException("Mark is not yet set");
        }
        return mark;
    }

    public void setMark(double mark) {
        this.mark = mark;
    }
}
```

CourseRecord.java

```
package latihan;

import java.util.Enumeration;
import java.util.Vector;

public class CourseRecord {
    private Student student;
    private Vector assignments;
    CourseRecord() {
        assignments=new Vector();
    }
    public Student getStudent() {
        return student;
    }
    public void setStudent(Student s) {
        student = s;
    }
    public void addAssignment(Assignment a) {
        assignments.addElement(a);
    }
    public Enumeration getAssignments() {
```

```
        return assignments.elements();
    }
}
```

Course.java

```
package latihan;

import java.util.Enumeration;
import java.util.Vector;

public class Course {
    private String title;
    private Vector courseRecords;

    Course() {
        courseRecords=new Vector();
    }

    public String getTitle() {return title;}
    public void setTitle( String t) {title=t;}
    public void addCourseRecord(CourseRecord cr) {
        courseRecords.addElement(cr);
    }

    public Enumeration getCourseRecords() {
        return courseRecords.elements();
    }

    public void printCourseStudents(){
        /*
         */
    }

    public void printBestStudent(){
        /*
         */
    }

    public void printFinalExamStudents(){
        /*
         */
    }

}
```

I.1.B. Analisa

Implementasi kode program di atas mencerminkan transformasi struktural dari rancangan Class Diagram ke dalam bahasa pemrograman Java dengan menerapkan prinsip-prinsip utama Pemrograman Berorientasi Objek (PBO). Struktur kelas dibangun secara hierarkis dan saling berelasi, di mana kelas Course bertindak sebagai kontainer utama yang mengelola kumpulan objek CourseRecord menggunakan struktur data Vector. Relasi asosiasi dan komposisi terlihat jelas pada kelas CourseRecord yang berfungsi sebagai penghubung antara entitas Student dengan daftar Assignment milik siswa tersebut, merepresentasikan hubungan one-to-many yang terdefinisi dalam desain. Selain itu, prinsip enkapsulasi diterapkan secara ketat melalui penggunaan access modifier private pada atribut vital (seperti mark, student, dan title), yang mewajibkan akses data dilakukan melalui metode accessor (getter) dan mutator (setter) guna menjaga integritas data.

Dari sisi perilaku (behavior), kode ini menunjukkan penerapan defensive programming yang baik melalui mekanisme penanganan kesalahan (exception handling). Pembuatan custom exception bernama NotYetException pada kelas Assignment memastikan bahwa nilai (mark) tidak dapat diambil sebelum diinisialisasi, sehingga mencegah terjadinya kesalahan logika saat runtime. Lebih lanjut, kelas Course telah menyediakan kerangka metode (method stubs) untuk pelaporan seperti printBestStudent dan printFinalExamStudents. Hal ini menunjukkan bahwa implementasi desain telah mempersiapkan struktur untuk logika bisnis yang lebih kompleks, memudahkan pengembangan fitur lanjutan tanpa perlu merombak arsitektur objek yang sudah terbentuk.

BAB II. TUGAS PRAKTIKUM

II.1 Implementasi Class Diagram

II.1.A. Source code

Circle.java

```
package tugas;

public class Cirlce extends Window{
    private float radius;
    private int center;
    public double area(float radius){
        return radius;
    }
    public void circum(){}

    public void setRadius(float radius) {
        this.radius = radius;
    }

    public void setCenter(int center) {
        this.center = center;
    }
}
```

ConsoleWindow.java

```
package tugas;

public class ConsoleWindow extends Window{

}
```

DataController.java

```
package tugas;

public class ConsoleWindow extends Window{

}
```

DialogBox.java

```
package tugas;

public class DialogBox extends Window{
}
```

DrawingContext.java

```
package tugas;

public class DrawingContext {
    public void setPoint(){

    }
    public void clearScreen(){

    }
    public void getVerticalSize(){

    }
    public void HorizontalSize(){

    }

}
```

Event.java

```
package tugas;

public class Event {

}
```

Frame.java

```
package tugas;

public class Frame {

}
```

Point.java

```
package tugas;  
  
public class Point {  
}
```

Polygon.java

```
package tugas;  
  
public class Polygon extends Shape{  
}
```

Rectangle.java

```
package tugas;  
  
public class Rectangle extends Shape{  
}
```

Shape.java

```
package tugas;  
  
public abstract class Shape {  
    public void draw(){  
  
    }  
  
    public void erase(){  
  
    }  
  
    public void move(){  
  
    }  
  
    public void resize(){  
  
    }  
}
```

Window.java

```
package tugas;

public class Window extends Frame {
}
```

II.1.B. Analisis

Implementasi kode program di atas mencerminkan transformasi struktural dari rancangan Class Diagram ke dalam bahasa pemrograman Java dengan menerapkan prinsip-prinsip utama Pemrograman Berorientasi Objek (PBO). Struktur kelas dibangun secara hierarkis dan saling berelasi, di mana kelas Course bertindak sebagai kontainer utama yang mengelola kumpulan objek CourseRecord menggunakan struktur data Vector. Relasi asosiasi dan komposisi terlihat jelas pada kelas CourseRecord yang berfungsi sebagai penghubung antara entitas Student dengan daftar Assignment milik siswa tersebut, merepresentasikan hubungan one-to-many yang terdefinisi dalam desain. Selain itu, prinsip enkapsulasi diterapkan secara ketat melalui penggunaan access modifier private pada atribut vital (seperti mark, student, dan title), yang mewajibkan akses data dilakukan melalui metode accessor (getter) dan mutator (setter) guna menjaga integritas data.

Dari sisi perilaku (behavior), kode ini menunjukkan penerapan defensive programming yang baik melalui mekanisme penanganan kesalahan (exception handling). Pembuatan custom exception bernama NotYetException pada kelas Assignment memastikan bahwa nilai (mark) tidak dapat diambil sebelum diinisialisasi, sehingga mencegah terjadinya kesalahan logika saat runtime. Lebih lanjut, kelas Course telah menyediakan kerangka metode (method stubs) untuk pelaporan seperti printBestStudent dan printFinalExamStudents. Hal ini menunjukkan bahwa implementasi desain telah mempersiapkan struktur untuk logika bisnis yang lebih kompleks, memudahkan pengembangan fitur lanjutan tanpa perlu merombak arsitektur objek yang sudah terbentuk.

.

BAB III. KESIMPULAN

Praktikum ini telah berhasil mendemonstrasikan proses transformasi desain perangkat lunak dari bentuk visual (Class Diagram) menjadi implementasi kode nyata menggunakan bahasa pemrograman Java. Melalui studi kasus pengelolaan nilai mahasiswa, kami mempelajari bahwa setiap elemen dalam desain—mulai dari atribut, metode, hingga hubungan antar kelas—memiliki padanan sintaks yang spesifik dalam kode. Hal ini menegaskan pentingnya tahap perancangan yang matang sebelum implementasi, di mana Class Diagram berfungsi sebagai cetak biru (blueprint) yang memandu struktur program agar terorganisir, logis, dan sesuai dengan spesifikasi kebutuhan sistem.

Penerapan prinsip-prinsip utama Object Oriented Programming (OOP) terlihat jelas dalam mekanisme Enkapsulasi dan relasi antar objek. Dengan menggunakan access modifier `private` pada atribut vital dan menyediakan metode `public` (getter/setter) sebagai pintu akses, integritas data tetap terjaga dari modifikasi yang tidak sah. Selain itu, implementasi relasi `association` dan `composition` menggunakan struktur data `Vector` pada kelas `Course` dan `CourseRecord` membuktikan efisiensi OOP dalam menangani hubungan data yang kompleks (one-to-many), memungkinkan satu objek untuk mengelola kumpulan objek lain secara dinamis dan terstruktur.

Terakhir, praktikum ini menyoroti pentingnya Defensive Programming untuk meningkatkan ketangguhan (robustness) aplikasi. Implementasi custom exception `NotYetException` menunjukkan bagaimana sistem harus secara proaktif menangani kondisi data yang belum siap atau tidak valid, mencegah terjadinya runtime error yang fatal. Kombinasi antara struktur kelas yang modular, perlindungan data melalui enkapsulasi, dan penanganan kesalahan yang tepat menyimpulkan bahwa implementasi desain berorientasi objek tidak hanya menghasilkan kode yang fungsional, tetapi juga kode yang aman, mudah dibaca, dan mudah dipelihara (maintainable) untuk pengembangan lebih lanjut.